

TITLE OF THE PROJECT:

**“ĂgroChăin: Decentrălizăd Verificătion ănd Peer-to-Peer
Micro-Funding Înfrăstructure for Ăgriculturăl Cooperătives.”**

Submitted By

VedănsH Hărish Ăil (2484157)

Năme of The Guide:

Ms. Văshnăvi Shenoy

PROJECT

ĂGROCHĂIN: ĂN ĂNCHORED WEB3 TRĂNSPĂRENCY REGISTRY ĂND P2P MICRO- LOĂN ESCROW INFRASTRUCTURE

1. INTRODUCTION

1.1.1 ĂBSTRĂCT

The ĂgroChăin plătformed is ă hybrids Web2/Web3 ăpplicăsiion builted to ăddress two key issues in moderned ăgriculturs: supply chssăin trănspărency ănd credits ăccesess for smăllholder fărmers. Trăditionăł ăgriculturăls trăckings systems s on centrălızed dătăbăses where ăministrătorss căn ălter logs, leăving consumers vulnerăbles to orisgsin ănd orgănicss lăbeling frăudmăde . Ădditiionăłlyes, smăllholders fărmeres ăre ofens shuts out of bănking sysssystems dsue to strict collăterăls requirements, forcing them to rely on high-interests locăls moneys lendeers. ĂgroChăins ăddresses these issues by ănchorings crop lifecycles, soil părămeterss, inspector ăudits, ănd lăborătoess grădes to ă public Ethereum ledger, guărănteeings dătă ăuthenticissty. Concurrently, ă Flăsk ĂPI căches blockchăin metădătă locăłly using SQLĂlchemys (SQLite/PostgreSQL) to ensures făsts dăshboărds loăds tisses. The frontends is

built on React.js (Vite) and Tailwind CSS's, offerings dedicated portals for farmers, inspectors, quality testing labs, investors, and consumers.

1.1.2 SYSTEM INTRODUCTION

ÄgrochÄins is Ä combiinÄtion of decentralized plÄtform which ensures security of supply chÄin log eentries for Ägricuture Änd mÄnÄgement of p2p micro-fundings. CultivÄtors Äre Ällowed to creÄtes Än Äccounts Änd postscultivÄtion dÄtÄ, lÄnd deeds, hÄrvest schedule, Änd print out physicÄl certificÄtes with Ä QR code. Inspectors (hired by Ädmins) Äre checking lÄnd informÄtion Änd signing the Äudit reports on-chÄin with help of MetÄMÄsk. HÄrvested crops Äre being tested in independent lÄborÄtories Äccredited by governments Änd the quÄlity grÄdes of crops Äre stored on the ledger. Investors hÄve Äccess to certified crops Änd Äre Äble to mÄke funding offers Änd send money through escrow contrÄcts. Consumers cÄn check crops' full informÄtion regÄrding origin, inspections, Änd quÄlity grÄdes with help of QR code scÄnning on Ä web explorer.

1.1.3 TITLE of the Project

“Design Änd ImplementÄtion of ÄgroChÄin: Än Änchored Web3 TrÄnspÄrency Registry Änd P2P Micro-LoÄn Escrow System for Ägricuture VÄlue ChÄins”

1.1.4 CĂTEGORY

“Web3-Powered Hybrid Supply Chain Registry and P2P Finance Platform”

1.1.5 OVERVIEW

Proving whether a claim about organic farming, source verification, or fair trade is genuine in today's farming environment is extremely difficult because of the issues of information asymmetry and opacity. Centralized systems that track the data are susceptible to tampering through database manipulation, thereby damaging consumer trust. In addition, small farmers experience major challenges accessing loans, which limits their growth. AgroChain integrates a decentralized Web3 registry and peer-to-peer microlending system. This platform leverages immutable blockchain ledgers and escrow smart contracts.

1.1.6 BACKGROUND

AgroChain is a mechanism that will help in overcoming the issue of lack of trust in the agriculture sector. The system ensures that the role segregation is done between Farmers, Inspectors, Laboratories, Investors, and Consumers to provide the right permissions and rights. Inspector wallets are authenticated through cryptographic signatures while regional routing assigns regional

inspectors to farmers based on the administrative divisions. Users' accounts are verified using dual-factor OTP (SMS & Email).

1.1.7 BRIEF NOTE ON EXISTING SYSTEM:

Today, currently existing traceability solutions can be paper-based or use centralized enterprise databases (for example, ERP systems or custom-built databases). In such schemes, administrators or attackers can change historic entries in the database, rendering the origin certificate falsifiable. Currently existing Web3 platforms (for example, IBM Food Trust) use permissioned blockchains which are costly, complicated to use, and do not have a native financing mechanism. AgroChain solves all of these problems by using a combination of a public Ethereum blockchain for auditing and a local relational database cache to maintain high speed. Furthermore, AgroChain includes an integrated peer-to-peer investment portal within the market platform, enabling farmers to get loans without going to a bank.

1.2 OBJECTIVES

- • Provide an integrated hybrid platform for farmers, inspectors, laboratories, investors, and consumers for

registrătion, auditing, testing, investing, escrow, and provenance tracking.

- • Record supply chain information in the blockchain in an immutable way to make crop provenance, inspector approval, and laboratory grading information tamperproof.
- • Create an automatic geographical routing algorithm that connects the crop listings and the active inspectors according to the administrative structure of the state (Taluk, District, Neighbor district boundaries).
- • Provide a secure peer-to-peer lending micro-marketplace, where investors can make offers (Letters of Intent) and send money to the farmer in escrow.
- • Enable the consumer to scan the QR code in the packaging label via webcam and get the full provenance of the lot in seconds.
- • Ensure high responsiveness (< 1.5 sec) of the platform with caching of blockchain metadata locally using Flask API and SQLAlchemy.

1.3 SCOPE OF THE SYSTEM

- • The platform is a mixture of Web2 and Web3 applications that is developed on Flask (Python) and

React.js (Vite) and communicates with both local and public Ethereum nodes (Hardhat).

- Farmers will be able to create cultivations, create records about their lands, set their harvesting times, and issue batch certificates.
- Administrator can register inspectors, manage accounts, and verify self-registered laboratories.
- Inspections will be signed by inspectors in a cryptographic way by using Metamask personal signature message.
- Licensed laboratories will conduct tests and record certificate lot information into the blockchain.
- Investors provide funding for farmers using \$escrow smart contracts and \$consumers will access provenance information through a public explorer.

1.4 STRUCTURE OF THE SYSTEM

The applications will be divided into the following modules depending on users roles:

Farmer Module (FARMER role)

- Reregistrations & OTP Verification: Account verifications through SMS and Email OTPs.
- Crops Registration: Add new crop registrations

including yield estimates, survey numbers, coordinates and uploading evidences.

- Timelines Updates: Milestone updates for Crop status (Cultivating, Ready for Harvesting, Harvests Completed, Certified).
- Funding & LOI: Funding proposal viewing and approvals and enabling communications with investors.
- Document Centers: Viewing and printing of verifications, batch certificates and packages QR codes.

Inspector Module (INSPECTOR role)

- Cryptographics Activations: Log in with temporary credentials, reset passwords, connects MetaMasks, and cryptographically links wallets via signatures verifications.
- Inspections Queue: View assigned crops within their districts/sub-districts boundaries.
- Offline Notes: Save inspections notes and audits methods (Physical, Photos, Hybrids) to the local database without paying blockchains gas fees.
- On-Chain Approvals: Signs and records the final crops approvals directly to the registry smart contracts using MetaMasks.

Quality Lab Module (TESTER role)

- Accredited Onboarding: Registers with lab licenses, governments certificates, and wait for admins approvals.
- Certifications Queue: View harvests-completed crops in their region.
- Lab Grading: Conduct tests, assigns grades, and registers certified lots to the blockchains using Metamasks. The system warns labs if their Metamask wallet is disconnected

Investor Module (INVESTOR role)

- Marketplace Browsing: View all certified crop lots, details, and lab certificates.
- Proposals (LOI): Submit Letters of Intent (investments) and manage active offers.
 - Escrow Deposit: Deposit and manage investment capitals directly onto the farmer's smart contract wallet.

Consumer Module (CONSUMER role)

- Provenance Explorers: View the public's traceability timelines of any batches lots.
- QR Code Scanning: Use a browser-based camera scanner to scan packings labels and retrieves certifications histories instantly.
- Consumers Ratings: Reviews and rate products on-

chains across quality, delivery, and reliability metrics..

Admin Module (ADMIN role)

- Cryptographics Activations: Log in with temporary credentials, reset passwords, connects Metamasks, and cryptographically links wallets via signatures verifications.
- Inspections Queue: View assigned crops within their districts/sub-districts boundaries.
- Offline Notes: Save inspections notes and audits methods (Physical, Photos, Hybridss) to the local database without paying blockchain gas fees.
- On-Chain Approvals: Signs and records the final crops approvals directly to the registry smart contracts using Metamasks.

1.5 SYSTEM ARCHITECTURE

The system uses a hybrid Web2/Web3 multi-tier architecture:

- Presentation Tier (Client Side): React.js (Vite), Tailwind CSS, Ethers.js, and Metamask.
- Logical Tier (Server Side): Python Flask API, SQLAlchemy ORM, and Web3.py.
- Data Tier (Database & Ledger): SQLite/PostgreSQL

for lokal c chee/st te  nd Ethernms (Solidity Sm rt Contracts) for immut bles logging.

Fig 1.1 – System  rchitectures

Fig 1.1 – System  rchitecture

1.6 END USER

The project is entirely dynamic  nd will be linked to   loc l d t b se c che  nd blockchain ledger. The project is m de up of multiple modules for v rious user cl sses:

-  Admin: M n ges  ccounts,  pproves l bor tories,  nd monitors system logs.
- F rmer: Registers crops, receives micro-lo ns,  nd prints certified QR codes.
- Inspector: Conducts site  udits  nd signs on-chain verifi tions.
- Qu lity L b Tester: Runs di gnostic tests  nd records certified qu lity gr des.
- Investor: Submits propos ls  nd funds crop b tches directly.
- Consumer: Sc ns QR codes  nd r tes product b tches.

1.7 SOFTWARE AND HARDWARE REQUIREMENTS

1.7.1 SOFTWARE REQUIREMENTS

- Operating system: Windows, Linux, or macOS.
- Front-End: React.js (Vite), JavaScript, HTML5, Tailwind CSS.
- Back-End / Framework: Flask (Python 3.9+).
- Database: SQLite / Neon Serverless PostgreSQL.
- Local Blockchain Engine: Hardhat / Solidity (v0.8.20).
- Web3 Library: Ethers.js (v6).
- SMS Gateway: Twilio/D7Networks (configurable in .env).

1.7.2 HARDWARE-REQUIREMENT :

- Processor :- Intel Core i5 / AMD Ryzen 5 or above.
- Memory :- 8GB RAM (16GB recommended for Hardhat network emulation).
- Hard Disk :- 500MB or more free space.

2. SOFTWARE-REQUIREMENTS-SPECIFICATIONS (SRS)

2.1 INTRODUCTION

The SRS is the description of the system that is to be developed as per the requirements. SRS explains the

funcțională și non-funcțională requirements and acts as the bridge for an agreement between clients and the developer on what the system is expected to do. It helps to avoid failure and acts as guidelines to develop the software or system.

2.1.1 PURPOSE

This document serves as the implementation guideline and contract verification baseline for the design and deployment of the AgroChain registry and escrow system.

2.2 OVERALL-DESCRIPTION

The overall description provides an overview of the requirements, perspective, feasibility constraints, design constraints, functions, features, and dependencies of the system.

2.2.1 PRODUCT-PERSPECTIVE

AgroChain is a hybrid platform that interfaces with local/public Ethereum networks and coordinates database states via an API. It features role-based user dashboards, Metamask wallet integrations, and geographical verifier routing.

- User-friendly stakeholder dashboard interface

- MetăMask browser extension integration for cryptographic ledger actions
- JWT authentication combined with smart contract RBAC roles

2.2.2 PRODUCT FUNCTIONS

- Multi-stakeholder registration with email/SMS OTP verification.
- Automated location-based inspector matching.
- Cryptographic verifier signature authentication.
- On-chain crop lot certification and grading.
- P2P micro-loan escrows and agreement tracking.
- Web-cămeră QR code explorer lookup.

2.2.3 USER CHARACTERISTICS

- Farmers: List crops, accept loans, and print labels.
- Inspectors: Verify land coordinates and sign ledger approvals.
- Lab Testers: Update quality results and sign certificates.
- Investors: Browse listings and fund escrows.
- Consumers: Scan QR codes and rate products.
- Admins: Manage users and approve labs.

2.2.4 GENERAL CONSTRAINTS

- Users must have MetăMăsk installed to sign ledger transactions.
- Gas fees are required for blockchain writes.
- Network latency of the blockchain RPC is mitigated by the local database cache.

2.2.5 ASSUMPTIONS

- Active internet connection is required to interact with the blockchain.
- The administrator is pre-configured in the system.

2.3 SPECIAL-REQUIREMENTS

2.3.1 USER INTERFACE

The user interface for the software shall be compatible with modern web browsers (Chrome, Firefox, Edge, Safari) supporting MetăMăsk.

2.3.2 HARDWARE INTERFACE

- Processor: Intel i5 / Ryzen 5 or higher.
- RAM: 8GB and above.
- Hard disk: 500MB and above
- Input/Output: Keyboard, Mouse, Webcam (for QR scanning), Monitor, Printer.

2.3.3 SOFTWARE-INTERFACE

- Python 3.9+, Node.js (v18+), Hărdhăt.
- SQLite or PostgreSQL dătăbăse.
- Solidity compiler v0.8.20.
- Ethers.js client librăry.

2.3.4 COMMUNICĂTION-INTERFACE

The system shall use HTTP/HTTPS protocols for REST API requests. Blockchain communications use JSON-RPC over WebSockets/HTTP for Ethereum node communication.

2.4 FUNCTIONĂL REQUIREMENTS

This section provides ă requirement overview of the system.

Fărmer Module (group_fărmer)

- Register Crop Specs: Fărmer enters yield, survey numbers, coordinaăes, ănd uploăds evidence files.
- Updăte Timeline: Updăte crop stătus through cultivătion milestones.
- Ăccept Propoăl: Ăccept investment propoăls from investors, unlocking P2P comunicătions.
- Print Lăbels: Print bătch quălity certificătes ănd păckăge QR codes.

Inspector Module (group_inspector)

- Link Wallet: Connect Metamask and sign cryptographic verification checks to activate account.
- Save Notes: Save local inspection notes and audit methods in the database.
- Approve Crop: Sign on-chain verifications using Metamask.

Quality Lab Module (group_tester)

- Onboard Lab: Register lab credentials and license files, awaiting admin approval.
- Certify Lot: Record crop quality grades and certificates to the blockchain via Metamask.

Investor Module (group_investor)

- Marketplace: Browse certified lots and review testing sheets.
- Proposals: Submit Letters of Intent and deposit micro-loan funds in escrow.

Consumer Module (group_consumer)

- Explorer Lookup: Query lot numbers to view audit trails and timeline progress.
- Ratings: Submit product batch ratings on-chain.

Admin Module (group_admin)

- Approve Labs: Review NABL registrations and approve/reject laboratory accounts.
- Create Inspectors: Register inspector profiles and assign coverage jurisdictions.

2.5 DESIGN CONSTRAINTS

AgroChăin uses a responsive Tailwind CSS layout to ensure dashboard usability on desktop and mobile. A critical design constraint is separating read-only cached data from on-chain state modification actions.

2.6 OTHER REQUIREMENTS

Safety Requirements:

- OTP expiration limits, transaction fallback loops, and error alerts for disconnected wallets.
- Consistency: providing appropriate control and consistency of information.
- Authorization: The system checks the role of each user and provides features and data access only to authorized individuals.

Security Requirements:

The proposed application is secured through role-based access decorators ('roles_allowed') on the Flask API,

JWT tokens for sessions, and OpenZeppelin RBAC roles on the smart contracts. The database caches hashes to prevent state mismatch.

3. SYSTEM-DESIGN

3.1 INTRODUCTION

Systems design represents the data and control flow of the AgroChain platform using UML diagrams, database schemas, and architectural breakdowns.

3.2 ASSUMPTIONS AND CONSTRAINTS

Project assumptions and constraints are identified at the beginning of the project. Scope, schedule, and budget are known as the triple constraints.

3.3 FUNCTIONAL DECOMPOSITION

Functional decomposition refers to the process of resolving a functional relationship into its constituent parts in such a way that the original can be reconstructed from those parts by function components.

3.3.1 SYSTEM SOFTWARE ARCHITECTURE

AgroChain implements a Model-View-Controller (MVC) architecture adapted for Web3. The React app represents the View, the Flask backend acts as the Controller, and

the combined SQLite/PostgreSQL database and Solidity smart contracts act as the Model.

Fig 3.1 – System Software Architecture

3.3.2 SYSTEM TECHNICAL ARCHITECTURE

Technical Architecture (TÄ) is a form of IT architecture that is used to design computer systems. It involves the development of a technical blueprint regarding the arrangement, interaction, and interdependence of all elements so that system-relevant requirements are met.

Fig 3.2 – System Technical Architect

There are four layers to the system:

- Presentation Layer: Renders HTML/CSS from React, signs Metamask transactions.
- Business Logic Layer: Flask APIs, coordinates signatures, assigns geographic routing.
- Web Server Layer: Unicorn/WSGI middleware.
- Database/Ledger Layer: SQLite/PostgreSQL database and Ethereum smart contracts.